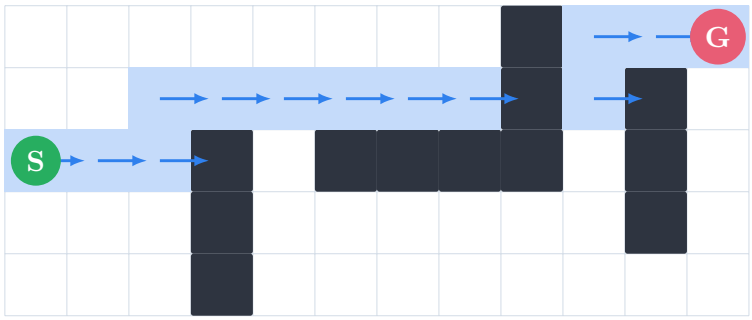
Dijkstra and A* for Robotics

A Gentle, Visual Path-Planning Guide

Made for learning from zero: concept → math → Python



A robot sees the world as states. Planning means choosing a sequence of states from start to goal.

Contents

1 Concept First: What Problem Are We Solving? 2
1.1 What Dijkstra Does . . . 2
1.2 What A* Does . . . 2

2 Robotics View: From Map to Graph 3

3 The Mathematics, Gently 3
3.1 Graph Model . . . 3
3.2 Dijkstra's Score . . . 4
3.3 A* Score . . . 4
3.4 Common Heuristics for Robot Grids . . . 5
3.5 Admissible and Consistent Heuristics . . . 5

4 Step-by-Step Dijkstra Example 5

5 Python Code: Dijkstra on a Graph 6

6 Python Code: Dijkstra on a Robot Grid 7

7 Python Code: A* on a Robot Grid 8

8 Seeing the Path in the Terminal 10

9 Dijkstra vs A*: What Should a Robotics Beginner Remember? 10

10 Exercises to Build Deep Understanding 10

11 The Core Ideas in One Page 11

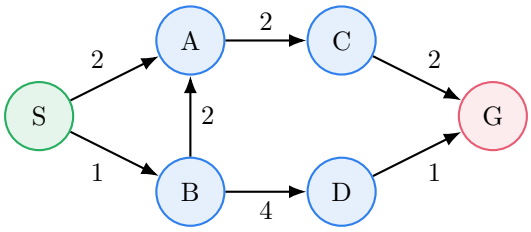
1 Concept First: What Problem Are We Solving?

The path-planning question

A robot is at a start position. It needs to reach a goal position. Some places are blocked, some motions are expensive, and the robot should choose a path that is valid and cheap.

In robotics, we often convert the world into a **graph**.

- A **node** is a possible robot state: a grid cell, a room, a road point, or even a full pose (x, y, θ).
- An **edge** is a possible movement from one state to another.
- A **cost** is how expensive that movement is: distance, time, energy, risk, or a mixture of these.



Nodes are states. Edges are actions. Numbers are movement costs.

1.1 What Dijkstra Does

Dijkstra's algorithm is like pouring water from the start. The water spreads outward in order of cheapest known distance. The first time the water reaches the goal, the route is guaranteed to be shortest, as long as all edge costs are non-negative.

Dijkstra in one sentence

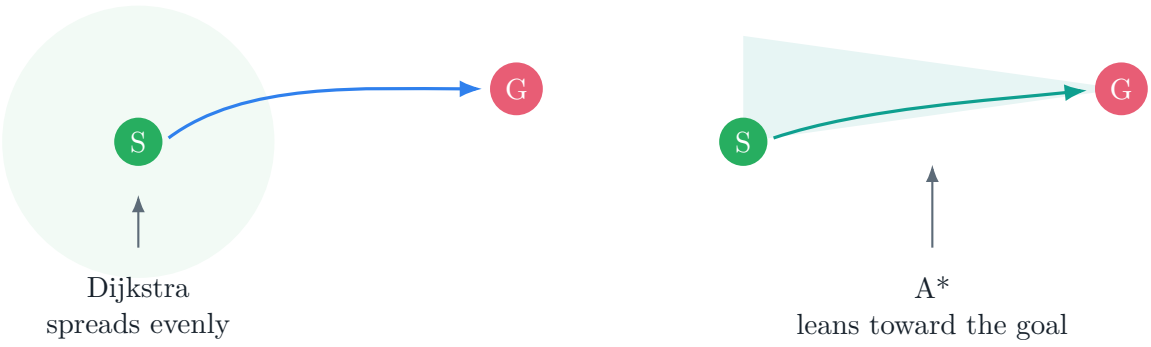
Always expand the not-yet-expanded node with the smallest known cost from the start.

1.2 What A* Does

A* is Dijkstra with a sense of direction. It still tracks the cost already paid, but it also estimates how far a node is from the goal. This estimate is called a **heuristic**.

A* in one sentence

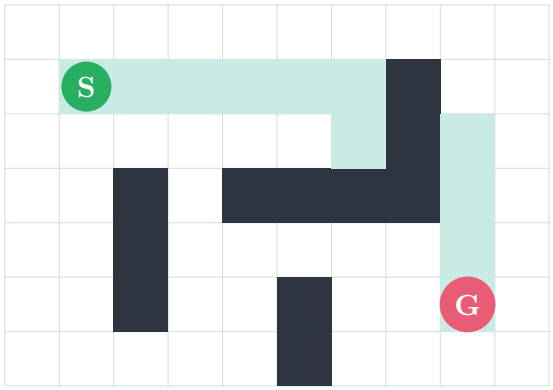
Always expand the node that looks cheapest by combining cost already paid and estimated cost still remaining.



2 Robotics View: From Map to Graph

For a beginner, the cleanest robotics example is a **grid map**. Imagine a small mobile robot moving on square cells.

- White cells are free.
- Dark cells are obstacles.
- The robot can move up, down, left, and right.
- Each move costs 1.



A grid map becomes a graph: every free cell is a node, every valid move is an edge.

This grid is a simplification. Real robots may need continuous motion, turning radius, collision checking, localization uncertainty, dynamic obstacles, and speed limits. But Dijkstra and A* remain foundational because many advanced planners are built on the same idea: search through possible states while keeping track of cost.

3 The Mathematics, Gently

3.1 Graph Model

We describe the planning problem as a weighted graph:

$$G = (V, E)$$

- V is the set of vertices, also called nodes or states.
- E is the set of edges, also called actions or transitions.
- Each edge (u, v) has a cost $w(u, v)$.

For shortest-path planning, we want:

$$\text{best path} = \arg \min_{\text{paths } P: s \to g} \sum_{(u,v) \in P} w(u, v)$$

In plain English: among all paths from start s to goal g, choose the path whose total movement cost is smallest.

3.2 Dijkstra's Score

Dijkstra keeps a value:

$$d(n) = \text{best cost found so far from start } s \text{ to node } n$$

At the start:

$$d(s) = 0, \qquad d(n) = \infty \text{ for every other node}$$

When Dijkstra sees that it can go from u to v, it asks:

$$\text{new cost to } v = d(u) + w(u, v)$$

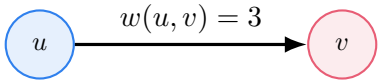
If this is better than the old value:

$$d(u) + w(u, v) < d(v)$$

then update:

$$d(v) \leftarrow d(u) + w(u, v)$$

This update is called **relaxation**. The word sounds fancy, but it simply means: "I found a cheaper way to reach this node."



$d(u) = 5$ $\qquad$ $d(v) = 10$

Candidate cost: $5 + 3 = 8$. Since $8 < 10$, update $d(v)$ to 8.

3.3 A* Score

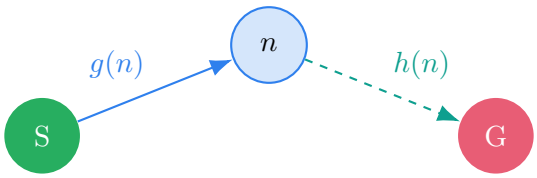
A* uses two costs:

$$g(n) = \text{cost already paid from start to } n$$

$$h(n) = \text{estimated cost from } n \text{ to the goal}$$

Then A* ranks nodes by:

$$f(n) = g(n) + h(n)$$



$$f(n) = g(n) + h(n)$$

paid cost + guessed remaining cost

3.4 Common Heuristics for Robot Grids

The heuristic should be easy to compute and should not overestimate the real cheapest remaining cost if you want guaranteed optimality.

Heuristic	Formula	Use when
Manhattan	$\lvert x_1 - x_2\rvert + \lvert y_1 - y_2\rvert$	Robot moves only 4 directions.
Euclidean	$\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$	Robot can move freely or diagonally.
Chebyshev	$\max(\lvert x_1 - x_2\rvert, \lvert y_1 - y_2\rvert)$	Diagonal moves cost the same as straight moves.

Why A* can be faster

Dijkstra asks, "What is the cheapest node I know how to reach?" A* asks, "What is the cheapest node if I include a reasonable guess toward the goal?"

3.5 Admissible and Consistent Heuristics

A heuristic is **admissible** if it never overestimates:

$$h(n) \leq h^*(n)$$

where $h^*(n)$ is the true shortest remaining cost from n to the goal.
A heuristic is **consistent** if:

$$h(u) \leq w(u, v) + h(v)$$

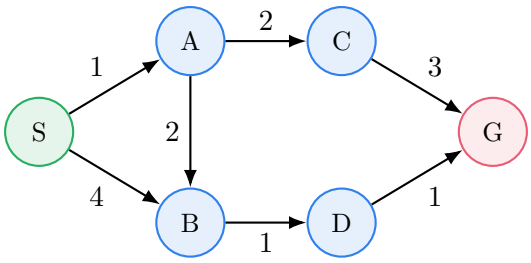
for every edge (u, v). Consistency means the heuristic behaves like a real distance: going from u to v and then estimating from v should not magically become more expensive than estimating from u directly.

Beginner mental model

If the heuristic is too optimistic, A* may explore extra nodes but still finds the best path. If the heuristic is too aggressive and overestimates, A* may become faster but can miss the true shortest path.

4 Step-by-Step Dijkstra Example

Consider this tiny graph:



Step	Expand	Known distances after update	Why
0	none	$d(S) = 0$, others ∞	Start has zero cost.
1	S	$d(A) = 1$, $d(B) = 4$	From S, A is cheaper than B.
2	A	$d(C) = 3$, $d(B) = 3$	B improves via A: $1 + 2 = 3$.
3	B	$d(D) = 4$	D reached with $3 + 1 = 4$.
4	C	$d(G) = 6$	G reached with $3 + 3 = 6$.
5	D	$d(G) = 5$	G improves via D: $4 + 1 = 5$.

The best path is:

$$S \to A \to B \to D \to G$$

with total cost:

$$1 + 2 + 1 + 1 = 5$$

5 Python Code: Dijkstra on a Graph

```python
import heapq


def dijkstra(graph, start, goal):
    """
    graph example:
    {
        "S": [("A", 1), ("B", 4)],
        "A": [("C", 2), ("B", 2)],
        ...
    }
    """
    frontier = [(0, start)] # (cost_so_far, node)
    came_from = {start: None}
    cost_so_far = {start: 0}

    while frontier:
        current_cost, current = heapq.heappop(frontier)

        if current == goal:
            break

        # Ignore old heap entries that are no longer the best.
        if current_cost > cost_so_far[current]:
            continue

        for neighbor, edge_cost in graph[current]:
            new_cost = cost_so_far[current] + edge_cost

            if neighbor not in cost_so_far or new_cost < cost_so_far[neighbor]:
                cost_so_far[neighbor] = new_cost
                came_from[neighbor] = current
                heapq.heappush(frontier, (new_cost, neighbor))

    return reconstruct_path(came_from, start, goal), cost_so_far.get(goal)


def reconstruct_path(came_from, start, goal):
    if goal not in came_from:
```

```
        return None

    path = []
    current = goal
    while current is not None:
        path.append(current)
        current = came_from[current]
    path.reverse()
    return path


graph = {
    "S": [("A", 1), ("B", 4)],
    "A": [("C", 2), ("B", 2)],
    "B": [("D", 1)],
    "C": [("G", 3)],
    "D": [("G", 1)],
    "G": [],
}

path, cost = dijkstra(graph, "S", "G")
print(path) # ['S', 'A', 'B', 'D', 'G']
print(cost) # 5
```

6 Python Code: Dijkstra on a Robot Grid

This version treats each free grid cell as a node. A cell is written as a coordinate:

$$(row, col)$$

```
import heapq


def neighbors_4(grid, cell):
    rows, cols = len(grid), len(grid[0])
    r, c = cell

    for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
        nr, nc = r + dr, c + dc

        inside = 0 <= nr < rows and 0 <= nc < cols
        if inside and grid[nr][nc] == 0:
            yield (nr, nc)


def dijkstra_grid(grid, start, goal):
    frontier = [(0, start)]
    came_from = {start: None}
    cost_so_far = {start: 0}

    while frontier:
        current_cost, current = heapq.heappop(frontier)

        if current == goal:
            break

        if current_cost > cost_so_far[current]:
            continue
```

```
        for neighbor in neighbors_4(grid, current):
            new_cost = cost_so_far[current] + 1

            if neighbor not in cost_so_far or new_cost < cost_so_far[neighbor]:
                cost_so_far[neighbor] = new_cost
                came_from[neighbor] = current
                heapq.heappush(frontier, (new_cost, neighbor))

    return reconstruct_path(came_from, start, goal), cost_so_far.get(goal)


def reconstruct_path(came_from, start, goal):
    if goal not in came_from:
        return None

    path = []
    current = goal
    while current is not None:
        path.append(current)
        current = came_from[current]
    path.reverse()
    return path


grid = [
    [0, 0, 0, 0, 0],
    [0, 1, 1, 1, 0],
    [0, 0, 0, 1, 0],
    [1, 1, 0, 0, 0],
]

start = (0, 0)
goal = (3, 4)

path, cost = dijkstra_grid(grid, start, goal)
print(path)
print(cost)
```

7 Python Code: A* on a Robot Grid

Only two things change from Dijkstra:

- We still store the real cost so far: $g(n)$.
- But the priority queue uses $f(n) = g(n) + h(n)$.

```
import heapq


def manhattan(a, b):
    ar, ac = a
    br, bc = b
    return abs(ar - br) + abs(ac - bc)


def neighbors_4(grid, cell):
    rows, cols = len(grid), len(grid[0])
```

```
    r, c = cell

    for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
        nr, nc = r + dr, c + dc

        inside = 0 <= nr < rows and 0 <= nc < cols
        if inside and grid[nr][nc] == 0:
            yield (nr, nc)


def astar_grid(grid, start, goal):
    frontier = [(0, start)] # (f_score, cell)
    came_from = {start: None}
    cost_so_far = {start: 0} # This is g(n)

    while frontier:
        current_f, current = heapq.heappop(frontier)

        if current == goal:
            break

        for neighbor in neighbors_4(grid, current):
            new_cost = cost_so_far[current] + 1

            if neighbor not in cost_so_far or new_cost < cost_so_far[neighbor]:
                cost_so_far[neighbor] = new_cost
                priority = new_cost + manhattan(neighbor, goal)
                came_from[neighbor] = current
                heapq.heappush(frontier, (priority, neighbor))

    return reconstruct_path(came_from, start, goal), cost_so_far.get(goal)


def reconstruct_path(came_from, start, goal):
    if goal not in came_from:
        return None

    path = []
    current = goal
    while current is not None:
        path.append(current)
        current = came_from[current]
    path.reverse()
    return path


grid = [
    [0, 0, 0, 0, 0],
    [0, 1, 1, 1, 0],
    [0, 0, 0, 1, 0],
    [1, 1, 0, 0, 0],
]

start = (0, 0)
goal = (3, 4)

path, cost = astar_grid(grid, start, goal)
print(path)
print(cost)
```

8 Seeing the Path in the Terminal

```python
def draw_grid(grid, path, start, goal):
    path_set = set(path or [])

    for r, row in enumerate(grid):
        line = []
        for c, value in enumerate(row):
            cell = (r, c)

            if cell == start:
                line.append("S")
            elif cell == goal:
                line.append("G")
            elif value == 1:
                line.append("#")
            elif cell in path_set:
                line.append("*")
            else:
                line.append(".")

        print(" ".join(line))


path, cost = astar_grid(grid, start, goal)
draw_grid(grid, path, start, goal)
print("cost:", cost)
```

Possible output:

```
S . . . .
* # # # .
* * * # .
# # * * G
cost: 7
```

9 Dijkstra vs A*: What Should a Robotics Beginner Remember?

Question	Dijkstra	A*
Uses heuristic?	No.	Yes.
Guaranteed shortest path?	Yes, with non-negative costs.	Yes, if heuristic is admissible and usually consistent.
Search behavior	Spreads outward from start.	Moves toward goal while respecting cost.
Good for	Unknown goal direction, all-pairs style thinking, baseline planner.	Goal-directed robot navigation.
Weakness	Can explore too much.	Needs a good heuristic.

10 Exercises to Build Deep Understanding

1. Change the grid and add more obstacles. Predict the path before running the code.
2. Print every node expanded by Dijkstra. Then print every node expanded by A*. Compare them.
3. Add diagonal movement. Use cost 1.414 for diagonal moves.

4. Replace Manhattan distance with Euclidean distance. Does the path change? Does the number of expanded nodes change?

5. Add terrain costs: road costs 1, grass costs 3, mud costs 6. Now the shortest path may not be the path with the fewest cells.

6. Try a bad heuristic: multiply Manhattan distance by 10. Watch how A* becomes greedy and may lose optimality.

11 The Core Ideas in One Page

Dijkstra

Store the cheapest known cost from the start to every node. Always expand the node with the smallest known cost. It is careful, reliable, and complete for non-negative edge costs.

A*

Store the cheapest known cost from the start, but rank nodes using:

$$f(n) = g(n) + h(n)$$

where $g(n)$ is the cost already paid and $h(n)$ is an estimate of the remaining cost. With a good heuristic, A* can find the same optimal path while expanding fewer nodes.

Robotics translation

A path planner is not magic. It repeatedly asks: "From where I am allowed to go next, which option is most promising according to my cost model?"